

---

# Detecting Fraud in the Graph

Graph neural networks + XGBoost, running entirely on one NVIDIA GB10

24.4 million card transactions · 589,000 scored per second · a regulator-grade explanation for every decision · no cloud, no network

# Four financial-services demos, all on-device

## Fraud detection

Graph neural network + XGBoost scoring card transactions in real time, with a Shapley explanation behind every decision.

589,528 txn/sec · F1 0.958

## Portfolio optimisation

Mean-CVaR optimisation on NVIDIA cuOpt — robust tail-risk allocation fast enough to iterate on, not batch overnight.

18x vs CPU · same optimum

## Document intelligence

Arabic financial document extraction into tables, text and a knowledge graph — the KYC and trade-finance back office.

on-device OCR + LLM

## Loan origination

A 7B vision-language model reading identity and income documents, with a state machine driving approval and deterministic cross-document fraud checks.

fine-tuned Qwen2.5-VL-7B

No cloud. No network at run time. One NVIDIA GB10 on the table.

## THE PROBLEM

# One fraud in every 819 transactions

---

**A model that says “never fraud” is 99.88% accurate — and worthless.**

In the IBM TabFormer dataset: 24,386,900 transactions, 29,757 of them fraudulent. A base rate of 0.122%.

**Extreme imbalance breaks conventional accuracy thinking.**

The cost of a miss and the cost of a false alarm are wildly asymmetric, and both are business decisions, not model settings.

**Fraud does not look unusual on its own row.**

The fraudulent transactions in this dataset are mostly ordinary amounts at ordinary times. Nothing in the row screams fraud.

**So the signal is not in the transaction. It is in the relationships around it.**

# The strongest fraud signals are entities, not amounts

Correlation with the fraud label, measured during preprocessing of all 24.4M rows:



**Where and with whom — not when, and not how much. That is a graph problem.**

# Model the transaction graph, then classify

---

## 1 Build the graph

Cardholders, transactions and merchants become nodes. Every payment becomes edges: user → transaction → merchant.

301,524 training transactions across 4,873 cardholders and 42,942 merchants.

## 2 Learn the neighbourhood

A GraphSAGE network aggregates each transaction's neighbourhood into an embedding — so a transaction is represented by the company it keeps, not just its own fields.

2 hops, 32 hidden channels.

## 3 Classify and explain

XGBoost scores the embeddings, and Shapley value sampling attributes each decision back to human-readable feature groups.

Every score arrives with its reasons.

Temporal split, not random: train before 2018, validate on 2018, test after. It is a backtest.

# What the booth screen shows



## THE MOMENT

# An ordinary amount, flagged at 99.5%



**Read the bars, not the numbers.**

Merchant city dominates the decision. Postal code follows. Transaction amount contributes under 1% as much — a sliver.

A rules engine watching for large or unusual amounts never sees this transaction.

**The fraud is not in the payment. It is in the company the payment keeps.**

MEASURED ON THE BOX

# Performance on a single NVIDIA GB10

**589,528**

transactions scored per second  
25,803 in 0.044 s

**0.958**

F1 score  
precision 94.3% · recall 97.3%

**41.7 s**

to preprocess 24.4M rows  
graph construction on GPU

**23.0 s**

to train end to end  
GNN + XGBoost

**3.3 s**

per live explanation  
Shapley, computed on demand

**9.4 s**

full power-cycle recovery  
container restart to serving

Portfolio optimisation: 18x vs CPU on a 51,502-variable LP. Document intelligence: full OCR, graph and vector pipeline on the same machine.



# A score a bank cannot justify is a score it cannot use

---

## **Every decision carries its reasons.**

Shapley attribution runs per transaction, not as a dataset-level feature-importance chart. “Why was this card declined?” has an answer.

## **Attribution is grouped into human language.**

Not 38 anonymous encoded columns — merchant city, postal code, entry mode, terminal errors, transaction amount.

## **It supports the conversation with the regulator and the customer.**

Adverse-action explanation, dispute handling, model risk documentation and audit all need the same thing: a defensible reason.

## **It also exposes the model's mistakes.**

The demo displays ground truth on every transaction, including its 123 false positives and 56 misses. Confidence, not concealment.

# A foundation model for spending



29M parameters. A 6,251-token financial vocabulary. Trained only to predict a cardholder's next transaction — never shown a fraud label.

## Fraud clusters anyway.

Adding these embeddings to the production fraud features lifted average precision by 26.9% — fewer false alarms at the same catch rate.

Embeddings alone score far worse than plain features. They complement feature engineering; they do not replace it.

59,123 transactions from a 1.2M corpus · projected with cuML UMAP in 2.2 s on the GB10

---

# Where this applies

The demo is card fraud. The architecture is a pattern for any problem where the answer lives in relationships.

## USE CASE 01

# Payment fraud at authorisation

---

Score every card transaction in the authorisation window, with the reason attached.

### WHAT CHANGES

Nothing — this is the demonstrated system.

### THE GRAPH

Cardholder → transaction → merchant.

### WHY GRAPH WINS

Catches ordinary-looking amounts at compromised merchants that amount-and-velocity rules miss.

### OPERATIONAL FIT

Sub-millisecond scoring leaves budget for the rest of the authorisation path.

**STATUS** Demonstrated today

# Money laundering and mule networks

---

Find the structure, not the suspicious individual: layering chains, mule rings, circular flows.

## WHAT CHANGES

Node types become accounts and counterparties; edges become transfers with amount and timing.

## THE GRAPH

Account → transfer → account, across institutions where data sharing permits.

## WHY GRAPH WINS

Mule networks are invisible per-account and obvious as a subgraph. This is the canonical GNN case.

## OPERATIONAL FIT

Batch scoring over a transfer window; explanations support the STR narrative.

**STATUS**    Strong architectural fit – not yet built

# Merchant and acquirer risk

---

Score the merchant, not just the payment: onboarding risk, transaction laundering, bust-out patterns.

## WHAT CHANGES

Predict on the merchant node instead of the transaction node.

## THE GRAPH

Merchant ↔ shared cardholders, terminals, beneficial owners, settlement accounts.

## WHY GRAPH WINS

Fraudulent merchants are identifiable by who they share customers and infrastructure with.

## OPERATIONAL FIT

Daily or weekly re-scoring; feeds portfolio monitoring and onboarding decisions.

**STATUS** Same node-prediction pattern

# Account takeover and synthetic identity

---

Detect accounts that behave like a network artefact rather than a person.

## WHAT CHANGES

Add device, session and identity-attribute nodes to the graph.

## THE GRAPH

Customer ↔ device ↔ session ↔ shared attributes (address, phone, employer).

## WHY GRAPH WINS

Synthetic identities are built from recombined real attributes — the reuse is only visible across accounts.

## OPERATIONAL FIT

Real-time at login and step-up; explanations justify the friction applied.

**STATUS** Requires additional data sources

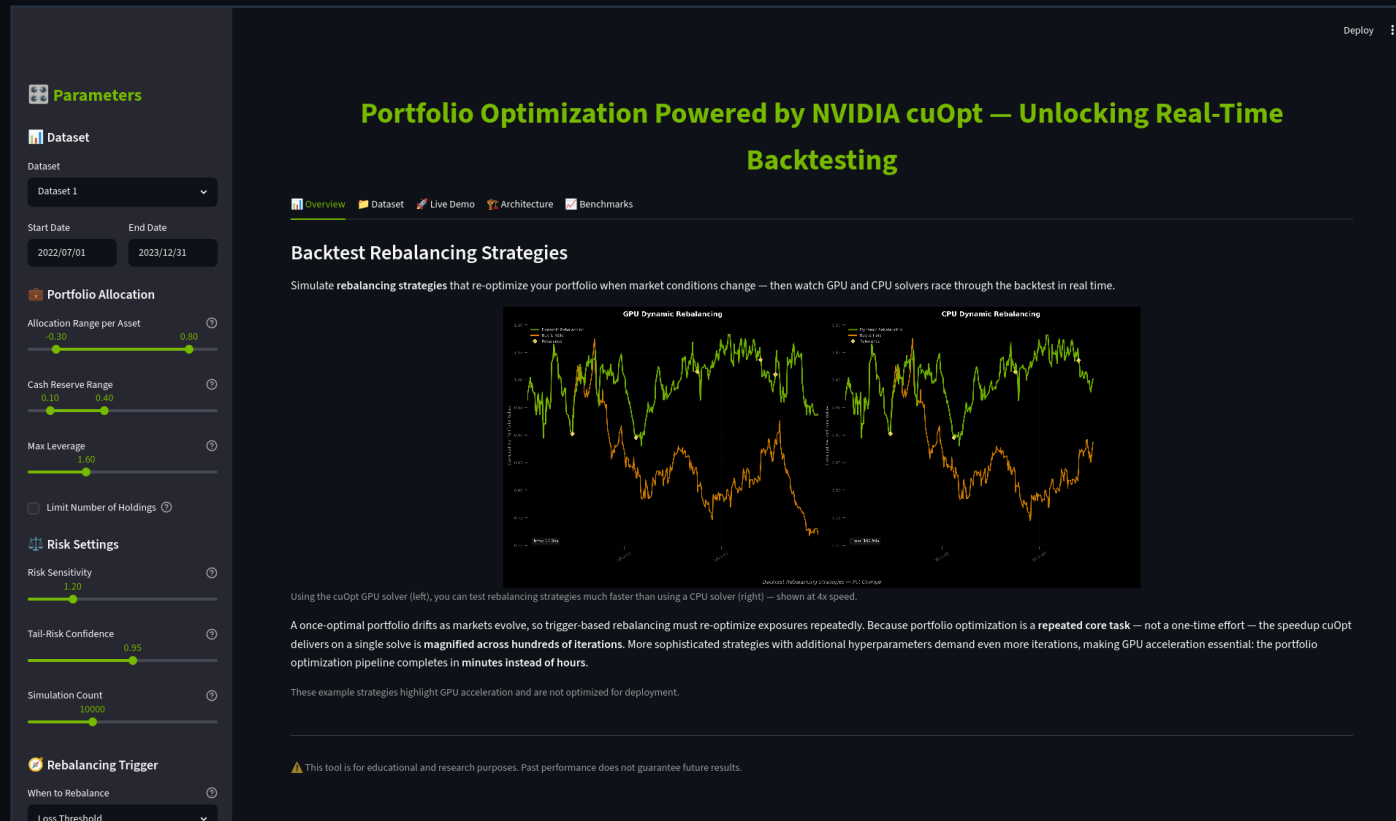
---

# Demo two — Portfolio optimisation

The same box, a different discipline: robust tail-risk allocation on NVIDIA cuOpt.



# Robust allocation, fast enough to iterate



Mean-CVaR asks a harder question than mean-variance: on the worst 5% of days, how much do I lose? Answering it means simulating tens of thousands of futures.

**11.7 s**

GPU · cuOpt

**211.7 s**

CPU · CVXPY

**18x**

faster, same optimum

500 assets × 50,000 scenarios — an LP with 51,502 variables.

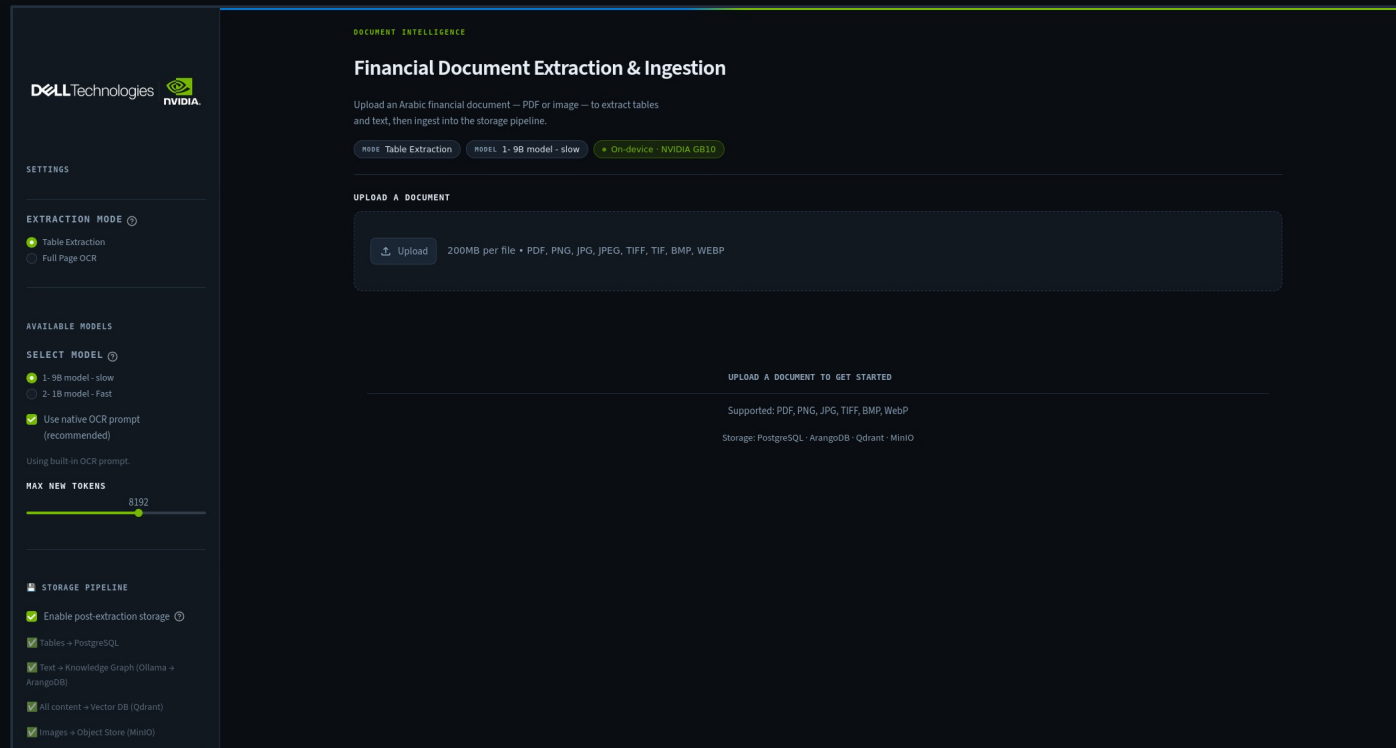
**Rebalancing is a repeated task, not a one-off. Across hundreds of backtest solves, that gap turns an overnight job into a morning's work.**

---

# Demo three — Document intelligence

Arabic financial documents into structured data — on the same box, still offline.

# Arabic documents into tables, text and a graph



KYC files, trade-finance paperwork and account-opening packs arrive as scans — often in Arabic, often as tables no rules engine can read.

## The pipeline

- Layout detection and OCR on the page
- Tables → PostgreSQL, structured and queryable
- Text → knowledge graph (Ollama → ArangoDB)
- All content → vector store (Qdrant) for retrieval
- Source images → object store (MinIO)

**Every model runs on the GB10. No document leaves the building.**

---

# Demo four — Loan origination

A vision-language model reading the documents a loan officer reads, and a state machine making the decision auditable.

# From document upload to an auditable decision

## 01 Capture

Applicant uploads identity and income documents.

## 02 Read

A fine-tuned 7B vision model reads each one natively — no OCR-then-parse, no per-template rules.

## 03 Cross-check

Deterministic validation across documents: name, employer, declared income.

## 04 Decide

A state machine walks the approval path and records why.

Fine-tuned on Egyptian documents:  
LoRA  $r=64$ , 4 epochs, eval loss 0.139,  
on Qwen2.5-VL-7B-Instruct.

The cross-document checks are arithmetic, not model output — so the model can be wrong without the decision being unsafe.

# Data residency without a cloud dependency

---

## **The entire system is on-device.**

Dataset, preprocessing, training, inference and explanation all run on a single GB10. During the demo the network cable is unplugged and nothing changes.

## **No transaction data leaves the institution — or the Kingdom.**

That removes the cross-border data-transfer question from the procurement conversation entirely, rather than answering it.

## **Deployable where the data already is.**

A desktop-class box fits in a branch, a data centre rack, or a regional processing site without redesigning the estate.

## **Recovery is measured in seconds.**

Full restart to serving in 9.4 seconds, with automatic restart on power loss.

# The caveats, stated first

---

## **These metrics come from a rebalanced benchmark, not production conditions.**

The held-out set runs at 8.09% fraud; the true base rate is 0.122%. At the real rate, false-positive economics change materially. Any pilot must be re-measured on live distributions.

## **Training undersamples the majority class.**

A 0.1 fraud ratio with computed class weights. Standard practice for extreme imbalance, and a deliberate modelling choice rather than a hidden one.

## **The dataset is public and synthetic-adjacent.**

IBM TabFormer, Apache 2.0. It demonstrates the method; it is not a proxy for any institution's own fraud patterns.

## **A production deployment needs the surrounding system.**

Case management, feedback loops, model monitoring, champion/challenger and drift detection are all out of scope for this demo.

# From demo to pilot

---

## Run it on your data.

The pipeline is three commands and roughly a minute of compute for a dataset this size. The graph schema adapts to your transaction model.

## Choose the use case with the clearest data.

Payment fraud and merchant risk need only data an issuer or acquirer already holds. AML and identity graphs need more sourcing.

## Measure at the true base rate from day one.

Agree the false-positive budget with the fraud operations team before tuning anything.

## Deploy where the data lives.

One GB10 per site, or scale up the same containers on larger NVIDIA infrastructure — the stack is identical.

Built on the NVIDIA AI Blueprint for Financial Fraud Detection · IBM TabFormer dataset (Apache 2.0) · running on NVIDIA GB10